Ejercicio 1

Quiero implementar en Java 21 un programa que ordene un arreglo de numeros enteros usando bubble sort. El programa recibe un arreglo desordenado y el algoritmo debe ir comparando de a pares elementos vecinos y si estan desordenados intercambiarlos, haciendo que en cada pasada completa el elemento mayor vaya desplazandose hacia el final del vector. Por pantalla se debe mostrar primero el arreglo original y despues imprimir el estado completo del arreglo luego de cada pasada del bucle externo para ver como va quedando. El codigo debe incluir comentarios claros que expliquen porque el elemento mayor se va desplazando hacia el final. Para verificar que el resultado sea correcto, se debe mostrar el arreglo ordenado de menor a mayor y chequear que no falte ningun numero.

Ejercicio 2

Necesito un programa en Java 21 que implemente dos metodos distintos para ordenar arreglos de enteros: bubbleSort(int[] array) y selectionSort(int[] array). Ambos algoritmos deben recibir una copia exacta del mismo arreglo desordenado. El razonamiento es diferente: burbuja compara vecinos todo el tiempo e intercambia seguido, mientras que seleccion busca directamente la posicion del elemento menor de todo el arreglo restante y hace un solo intercambio directo por cada ciclo. El programa debe llevar contadores y mostrar por pantalla cuantas comparaciones y cuantos intercambios realizo cada metodo. El codigo debe tener comentarios explicando el trade off y la diferencia entre comparar vecinos contra buscar el minimo. Se debe verificar imprimiendo los dos arreglos finales ordenados y viendo que la cantidad de intercambios en seleccion sea mucho menor que en burbuja.

Ejercicio 3

Quiero implementar en Java 21 el algoritmo insertion sort para ordenar un arreglo de enteros que ya viene casi ordenado de entrada, por ejemplo int[] numeros = {1, 2, 3, 5, 4, 6, 7}. El razonamiento del algoritmo es como ordenar cartas en la mano: mantiene una zona ordenada a la izquierda y va tomando el siguiente elemento para insertarlo corriendo los mayores hacia la derecha. Este metodo es muy eficiente sobre arreglos casi ordenados porque hace pocos desplazamientos logrando una complejidad temporal cercana a O(n). El programa debe contar y mostrar por pantalla la cantidad exacta de desplazamientos realizados y el arreglo final. Se debe verificar comprobando que no falte ningun numero en el vector ordenado.

Ejercicio 4

Se requiere implementar un algoritmo para Java 21 que ordene alfabeticamente de menor a mayor un arreglo de strings usando selection sort. El programa recibe por ejemplo String[] nombres = {"Lucia", "Ana", "Pedro", "Juan"}. El algoritmo busca en cada iteracion el nombre lexicograficamente menor del resto del vector y lo intercambia con la posicion actual. En los comentarios del codigo se debe explicar como cambia la comparacion al ordenar textos en lugar de numeros primitivos, ya que no se puede usar el operador < y es obligatorio utilizar el metodo .compareTo() de la clase String. Por consola se debe mostrar el arreglo de nombres original y el arreglo final ordenado alfabeticamente. Para verificar que funcione

bien hay que probarlo con nombres con mayusculas, minusculas y nombres que arranquen con la misma letra.

Ejercicio 5

Quiero implementar el algoritmo shellsort en Java 21 para ordenar un vector de numeros enteros. El programa recibe un arreglo desordenado de enteros y debe ir aplicando saltos llamados gaps (por ejemplo arrancando con gap = n/2 y dividiendo a la mitad en cada paso hasta llegar a gap = 1). Shellsort es una mejora de insertion sort, porque al hacer intercambios entre elementos lejanos con los gaps acomoda rapido los numeros fuera de lugar sin tener que correr de a una sola posicion. Por pantalla el programa debe mostrar el valor del gap en cada etapa y como queda el arreglo despues de cada pasada. El codigo debe estar comentado explicando la complejidad segun la secuencia de saltos y se verifica viendo que con gap = 1 termine de ordenar el arreglo completamente.

Ejercicio 6

Necesito implementar en Java 21 el algoritmo quicksort para ordenar un arreglo de enteros, tomando siempre el primer elemento del subarreglo como pivote. El algoritmo se basa en el paradigma divide y venceras: elige el pivote, particiona dejando los menores a la izquierda y los mayores a la derecha, y despues se llama recursivamente para ambos lados. El programa debe recibir un arreglo desordenado y mostrar en cada nivel de recursion: cual fue el pivote elegido, como quedo el subarreglo izquierdo, el subarreglo derecho y finalmente el arreglo ordenado completo. Se verifica imprimiendo el arreglo final ordenado y confirmando que no se pierdan elementos durante las particiones.

Ejercicio 7

Quiero implementar en Java 21 un programa que implemente quicksort en un arreglo ya ordenado, por ejemplo int[] numeros = {1, 2, 3, 4, 5, 6, 7} y usando siempre el primer elemento como pivote. Al estar ordenado, el pivote siempre resulta ser el elemento menor posible, generando particiones de tamaño 0 y n-1, lo que resulta en una complejidad temporal O(n^2). El programa debe contar e imprimir por pantalla la cantidad total de llamadas recursivas realizadas. Se verifica imprimiendo el arreglo final ordenado y confirmando que no se perdieron elementos.

Ejercicio 8

Se requiere implementar en Java 21 el algoritmo mergesort recursivo para ordenar un vector de enteros. El programa recibe un arreglo desordenado y aplica la estrategia de divide y venceras dividiendo el arreglo exactamente a la mitad sin usar pivotes hasta llegar a elementos individuales (condicion de corte). Dividir y fusionar son conceptos distintos. Dividir es cortar a la mitad en la recursion y la fase de fusion es mezclar dos mitades ordenadas usando memoria auxiliar. El programa debe mostrar paso a paso por pantalla: como se va partiendo el arreglo, cuando toca la condicion de corte y como se van fusionando los subarreglos hasta el resultado final. Se verifica revisando que el arreglo final quede ordenado y comentando el trade off del costo de memoria extra O(n) que requiere.

Ejercicio 9

Quiero implementar en Java 21 un programa que compare los tres algoritmos de ordenamiento bubble sort, selection sort e insertion sort. El programa debe recibir tres copias del mismo arreglo desordenado de numeros enteros para probar cada metodo por separado. El objetivo es comparar las estrategias de cada uno: vecinos contiguos en burbuja, busqueda del menor en seleccion e insercion progresiva en insercion. El programa tiene que registrar y mostrar por pantalla para cada algoritmo: el arreglo final ordenado, la cantidad total de comparaciones realizadas y la cantidad de intercambios o desplazamientos hechos. El codigo debe incluir comentarios analizando las diferencias de rendimiento entre ellos y se verifica viendo que los tres entreguen el mismo arreglo ordenado pero con distinta cantidad de operaciones.

Ejercicio 10

Necesito un algoritmo en Java 21 para armar un ranking de jugadores que ordene de mayor a menor segun su puntaje. El programa recibe un arreglo de objetos tipo Jugador con atributos nombre y puntaje, por ejemplo Jugador("Ana", 1200), Jugador("Pedro", 900) y Jugador("Lucia", 1500). Para resolverlo elijo implementar insertion sort porque en un sistema de ranking real los puntajes suelen llegar de a uno o estar casi ordenados, siendo muy eficiente y facil de mantener estable sin requerir demasiada memoria auxiliar. El programa debe imprimir por consola la tabla de posiciones final con el puesto, nombre y puntaje. Se verifica corroborando que el jugador con mas puntos quede primero y el de menor puntaje al final.